TABLE OF CONTENTS (HIDE)

1. Pointer Variables
  1.1 Pointer Variables (or Pointers)
  1.2 Declaring Pointers
  1.3 Initializing Pointers via the Ad
  1.4 Indirection or Dereferencing C
  1.5 Pointer has a Type Too
  1.6 Uninitialized Pointers
  1.7 Null Pointers
2. Reference Variables
  2.1 References (or Aliases) (&)
  2.2 How References Work?
  2.3 References vs. Pointers
  2.4 Pass-By-Reference into Func
  2.5 Passing the Function's Return
  2.6 Summary
3. Dynamic Memory Allocation
  3.1 `new` and `delete` Operators
  3.2 `new[]` and `delete[]` Opera
4. Pointer, Array and Function
  4.1 Array is Treated as Pointer
  4.2 Pointer Arithmetic
  4.3 `sizeof` Array
  4.4 Passing Array In/Out of a Fun
  4.5 Pass-by-Reference and `size`
  4.6 Operating on a Range of an A
  4.7 C-String and Pointer
5. *More On Pointers
  5.1 Function Pointer
  5.2 Generic Pointer or void Pointe
  5.3 Constant Pointer vs. Constan

C++ Programming Language

Pointers, References and Dynamic Memory Allocation

Pointers, References and Dynamic Memory Allocation are the most powerful features in C/C++ language, which allows programmers to directly manipulate memory to *efficiently manage the memory* - the most critical and scarce resource in computer - *for best performance*. However, "pointer" is also the most complex and difficult feature in C/C++ language.

Pointers are extremely powerful because they allows you to access addresses and manipulate their contents. But they are also extremely complex to handle. Using them correctly, they could greatly improve the efficiency and performance. On the other hand, using them incorrectly could lead to many problems, from un-readable and un-maintainable codes, to infamous bugs such as memory leaks and buffer overflow, which may expose your system to hacking. Many new languages (such as Java and C#) remove pointer from their syntax to avoid the pitfalls of pointers, by providing automatic memory management.

Although you can write C/C++ programs without using pointers, however, it is difficult not to mention pointer in teaching C/C++ language. Pointer is probably not meant for novices and dummies.

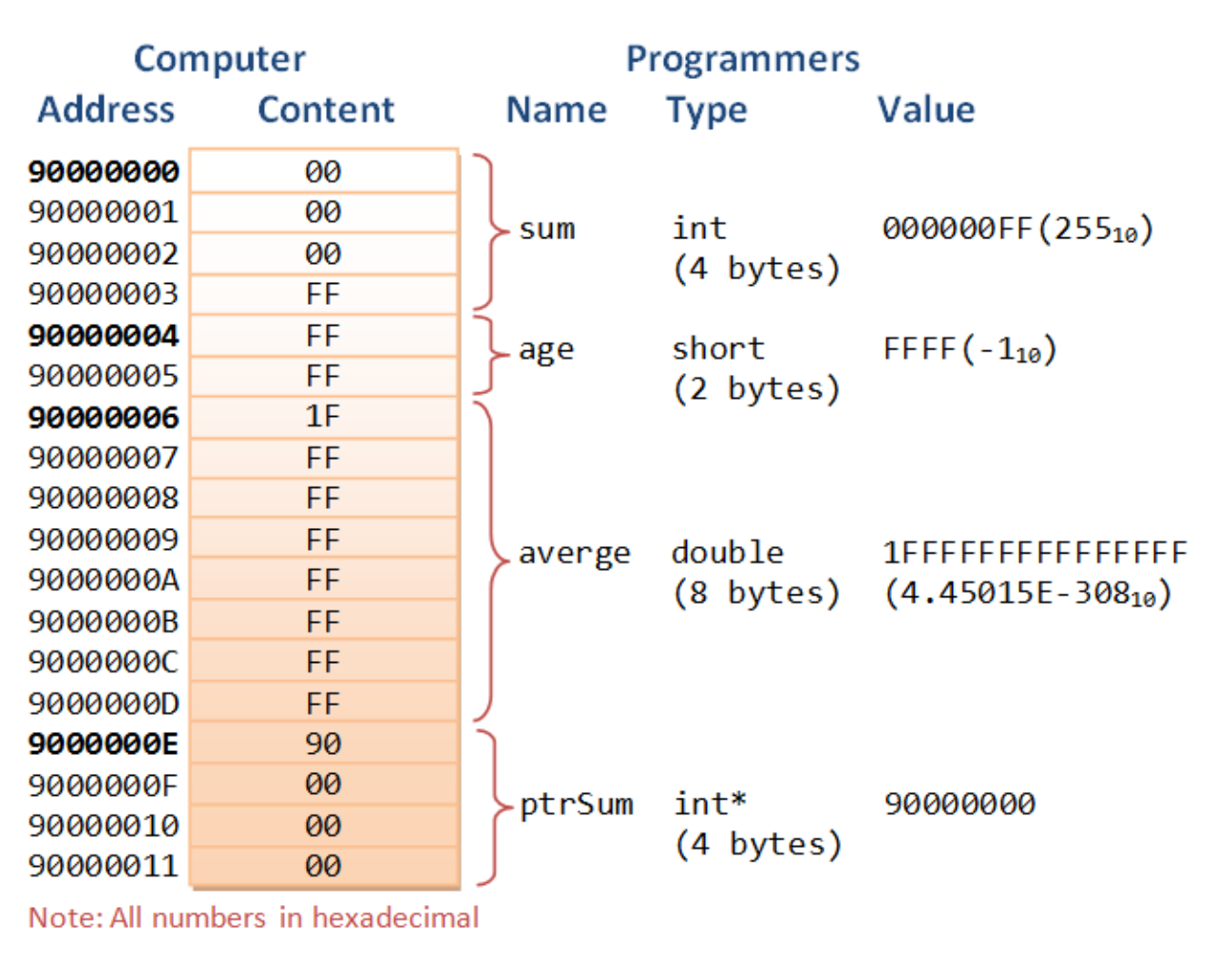
1. Pointer Variables

A *computer memory location* has an *address* and holds a *content*. The *address* is a numerical number (often expressed in hexadecimal), which is hard for programmers to use directly. Typically, each address location holds 8-bit (i.e., 1-byte) of data. It is entirely up to the programmer to interpret the meaning of the data, such as integer, real number, characters or strings.

To ease the burden of programming using numerical address and programmer-interpreted data, early programming languages (such as C) introduce the concept of variables. A variable is a *named* location that can store a *value* of a particular *type*. Instead of numerical addresses, names (or identifiers) are attached to certain addresses. Also, types (such as `int`, `double`, `char`) are associated with the contents for ease of interpretation of data.

Each address location typically hold 8-bit (i.e., 1-byte) of data. A 4-byte `int` value occupies 4 memory locations. A 32-bit system typically uses 32-bit addresses. To store a 32-bit address, 4 memory locations are required.

The following diagram illustrate the relationship between computers' memory *address* and *content*; and variable's *name*, *type* and *value* used by the programmers.



1.1 Pointer Variables (or Pointers)

A *pointer variable* (or *pointer* in short) is basically the same as the other variables, which can store a piece of data. Unlike normal variable which stores a value (such as an `int`, a `double`, a `char`), a *pointer stores a memory address*.

1.2 Declaring Pointers

Pointers must be declared before they can be used, just like a normal variable. The syntax of declaring a pointer is to place a `*` in front of the name. A pointer is associated with a type (such as `int` and `double`) too.

```
type *ptr;   // Declare a pointer variable called ptr as a pointer of type
// or
type* ptr;
// or
type * ptr;  // I shall adopt this convention
```

For example,

```
int * iPtr;     // Declare a pointer variable called iPtr pointing to an int (an int pointer)
                // It contains an address. That address holds an int value.
double * dPtr;  // Declare a double pointer
```

Take note that you need to place a `*` in front of each pointer variable, in other words, `*` applies only to the name that followed. The `*` in the declaration statement is not an operator, but indicates that the name followed is a pointer variable. For example,

```
int *p1, *p2, i;    // p1 and p2 are int pointers. i is an int
int* p1, p2, i;     // p1 is a int pointer, p2 and i are int
int * p1, * p2, i;  // p1 and p2 are int pointers, i is an int
```

Naming Convention of Pointers: Include a "`p`" or "`ptr`" as *prefix* or *suffix*, e.g., `iPtr`, `numberPtr`, `pNumber`, `pStudent`.

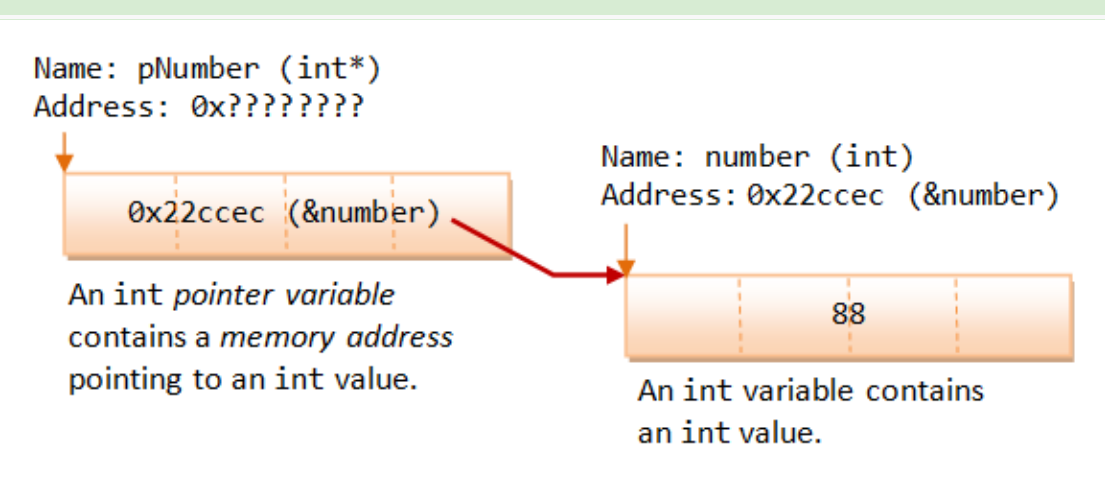
1.3 Initializing Pointers via the Address-Of Operator (&)

When you declare a pointer variable, its content is not initialized. In other words, it contains an address of "somewhere", which is of course not a valid location. This is dangerous! You need to initialize a pointer by assigning it a valid address. This is normally done via the *address-of operator* (`&`).

The *address-of operator* (`&`) operates on a variable, and returns the address of the variable. For example, if `number` is an `int` variable, `&number` returns the address of the variable `number`.

You can use the address-of operator to get the address of a variable, and assign the address to a pointer variable. For example,

```
int number = 88;     // An int variable with a value
int * pNumber;       // Declare a pointer variable called pNumber pointing to an int (or int pointer)
pNumber = &number;   // Assign the address of the variable number to pointer pNumber

int * pAnother = &number; // Declare another int pointer and init to address of the variable number
```


Name: pNumber (int*)
Address: 0x????????
0x22ccec (&number)
An int pointer variable contains a memory address pointing to an int value.
Name: number (int)
Address: 0x22ccec (&number)
88
An int variable contains an int value.

As illustrated, the `int` variable `number`, starting at address `0x22ccec`, contains an `int` value `88`. The expression `&number` returns the address of the variable `number`, which is `0x22ccec`. This address is then assigned to the pointer variable `pNumber`, as its initial value.

The address-of operator (`&`) can only be used on the RHS.

1.4 Indirection or Dereferencing Operator (*)

The *indirection operator* (or *dereferencing operator*) (`*`) operates on a pointer, and returns the value stored in the address kept in the pointer variable. For example, if `pNumber` is an `int` pointer, `*pNumber` returns the `int` value "*pointed to*" by `pNumber`.

For example,

```
int number = 88;
int * pNumber = &number;  // Declare and assign the address of variable number to pointer pNumber (0x22ccec)
cout << pNumber<< endl;   // Print the content of the pointer variable, which contain an address (0x22ccec)
cout << *pNumber << endl; // Print the value "pointed to" by the pointer, which is an int (88)
*pNumber = 99;            // Assign a value to where the pointer is pointed to, NOT to the pointer variable
cout << *pNumber << endl; // Print the new value "pointed to" by the pointer (99)
cout << number << endl;   // The value of variable number changes as well (99)
```

Take note that `pNumber` stores a memory address location, whereas `*pNumber` refers to the value stored in the address kept in the pointer variable, or the value pointed to by the pointer.

As illustrated, a variable (such as `number`) *directly* references a value, whereas a pointer *indirectly* references a value through the memory address it stores. Referencing a value indirectly via a pointer is called *indirection* or *dereferencing*.

The indirection operator (`*`) can be used in both the RHS (`temp = *pNumber`) and the LHS (`*pNumber = 99`) of an assignment statement.

Take note that the symbol `*` has different meaning in a declaration statement and in an expression. When it is used in a declaration (e.g., `int * pNumber`), it denotes that the name followed is a pointer variable. Whereas when it is used in a expression (e.g., `*pNumber = 99; temp << *pNumber;`), it refers to the value pointed to by the pointer variable.

1.5 Pointer has a Type Too

A pointer is associated with a type (of the value it points to), which is specified during declaration. A pointer can only hold an address of the declared type; it cannot hold an address of a different type.

```
int i = 88;
double d = 55.66;
int * iPtr = &i;    // int pointer pointing to an int value
double * dPtr = &d; // double pointer pointing to a double value

iPtr = &d;   // ERROR, cannot hold address of different type
dPtr = &i;   // ERROR
iPtr = i;    // ERROR, pointer holds address of an int, NOT int value

int j = 99;
iPtr = &j;  // You can change the address stored in a pointer
```

Example

```
/* Test pointer declaration and initialization (TestPointerInit.cpp) */
#include <iostream>
using namespace std;

int main() {
   int number = 88;    // Declare an int variable and assign an initial value
   int * pNumber;      // Declare a pointer variable pointing to an int (or int pointer)
   pNumber = &number;  // assign the address of the variable number to pointer pNumber

   cout << pNumber << endl;  // Print content of pNumber (0x22ccf0)
   cout << &number << endl;  // Print address of number (0x22ccf0)
   cout << *pNumber << endl; // Print value pointed to by pNumber (88)
   cout << number << endl;   // Print value of number (88)

   *pNumber = 99;            // Re-assign value pointed to by pNumber
   cout << pNumber << endl;  // Print content of pNumber (0x22ccf0)
   cout << &number << endl;  // Print address of number (0x22ccf0)
   cout << *pNumber << endl; // Print value pointed to by pNumber (99)
   cout << number << endl;   // Print value of number (99)
                             // The value of number changes via pointer

   cout << &pNumber << endl; // Print the address of pointer variable pNumber (0x22ccec)
}
```

Notes: The address values that you get are unlikely to be the same as mine. The OS loads the program in available free memory locations, instead of fixed memory locations.

1.6 Uninitialized Pointers

The following code fragment has a serious logical error!

```
int * iPtr;
*iPtr = 55;
cout << *iPtr << endl;
```

The pointer `iPtr` was declared without initialization, i.e., it is pointing to "somewhere" which is of course an invalid memory location. The `*iPtr = 55` corrupts the value of "somewhere"! You need to initialize a pointer by assigning it a valid address. Most of the compilers does not signal an error or a warning for uninitialized pointer?!

1.7 Null Pointers

You can initialize a pointer to 0 or `NULL`, i.e., it points to nothing. It is called a *null pointer*. Dereferencing a null pointer (`*p`) causes an `STATUS_ACCESS_VIOLATION` exception.

```
int * iPtr = 0;         // Declare an int pointer, and initialize the pointer to point to nothing
cout << *iPtr << endl;  // ERROR! STATUS_ACCESS_VIOLATION exception

int * p = NULL;         // Also declare a NULL pointer points to nothing
```

Initialize a pointer to null during declaration is a good software engineering practice.

C++11 introduces a new keyword called `nullptr` to represent null pointer.

2. Reference Variables

C++ added the so-called *reference variables* (or *references* in short). A reference is an *alias*, or an *alternate name* to an existing variable. For example, suppose you make `peter` a reference (alias) to `paul`, you can refer to the person as either `peter` or `paul`.

The main use of references is acting as function formal parameters to support pass-by-reference. In an reference variable is passed into a function, the function works on the original copy (instead of a clone copy in pass-by-value). Changes inside the function are reflected outside the function.

A reference is similar to a pointer. In many cases, a reference can be used as an alternative to pointer, in particular, for the function parameter.

2.1 References (or Aliases) (&)

Recall that C/C++ use `&` to denote the *address-of* operator in an expression. C++ assigns an additional meaning to `&` in declaration to declare a reference variable.

The meaning of symbol `&` is different in an expression and in a declaration. When it is used in an expression, `&` denotes the address-of operator, which returns the address of a variable, e.g., if `number` is an `int` variable, `&number` returns the address of the variable `number` (this has been described in the above section).

However, when `&` is used in a *declaration* (including *function formal parameters*), it is part of the type identifier and is used to declare a *reference variable* (or *reference* or *alias* or *alternate name*). It is used to provide *another name*, or *another reference*, or *alias* to an existing variable.

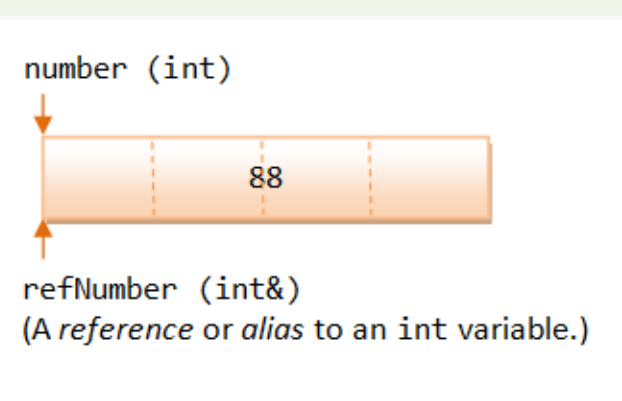
The syntax is as follow:

```
type &newName = existingName;
// or
type& newName = existingName;
// or
type & newName = existingName;  // I shall adopt this convention
```

It shall be read as "`newName` *is a reference to* `existingName`", or "`newNew` *is an alias of* `existingName`". You can now refer to the variable as `newName` or `existingName`.

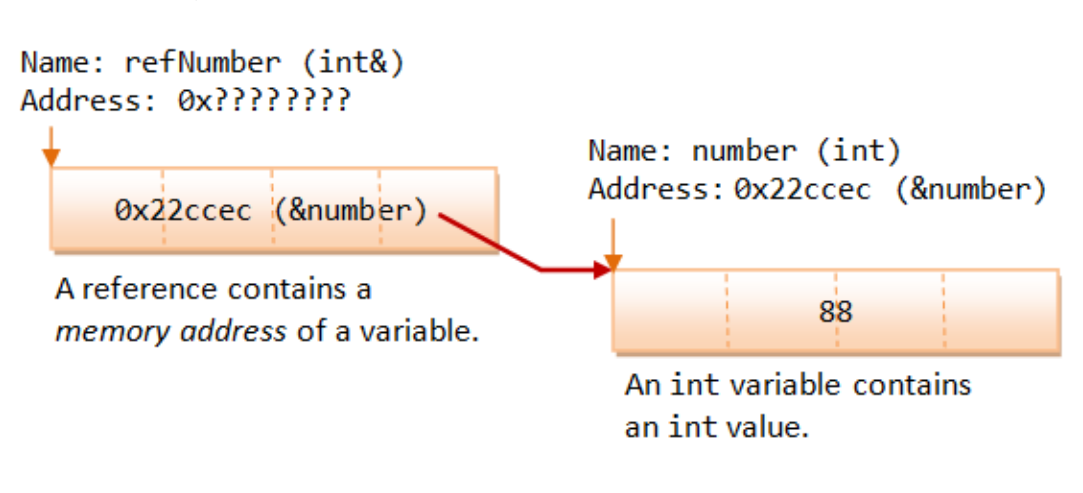
For example,

```
/* Test reference declaration and initialization (TestReferenceDeclaration.cpp) */
#include <iostream>
using namespace std;

int main() {
   int number = 88;          // Declare an int variable called number
   int & refNumber = number; // Declare a reference (alias) to the variable number
                             // Both refNumber and number refer to the same value

   cout << number << endl;    // Print value of variable number (88)
   cout << refNumber << endl; // Print value of reference (88)

   refNumber = 99;            // Re-assign a new value to refNumber
   cout << refNumber << endl;
   cout << number << endl;    // Value of number also changes (99)

   number = 55;               // Re-assign a new value to number
   cout << number << endl;
   cout << refNumber << endl; // Value of refNumber also changes (55)
}
```



2.2 How References Work?

A reference works as a pointer. A reference is declared as an alias of a variable. It stores the address of the variable, as illustrated:


Name: refNumber (int&)
Address: 0x????????
0x22ccec (&number)
A reference contains a memory address of a variable.
Name: number (int)
Address: 0x22ccec (&number)
88
An int variable contains an int value.

2.3 References vs. Pointers

Pointers and references are equivalent, except:

1. A reference is a *name constant for an address*. You need to initialize the reference during declaration.

   ```
   int & iRef;  // Error: 'iRef' declared as reference but not initialized
   ```

   Once a reference is established to a variable, you cannot change the reference to reference another variable.
2. To get the value pointed to by a pointer, you need to use the dereferencing operator `*` (e.g., if `pNumber` is a `int` pointer, `*pNumber` returns the value pointed to by `pNumber`. It is called *dereferencing* or *indirection*). To assign an address of a variable into a pointer, you need to use the address-of operator `&` (e.g., `pNumber = &number`).

   On the other hand, referencing and dereferencing are done on the references implicitly. For example, if `refNumber` is a reference (alias) to another `int` variable, `refNumber` returns the value of the variable. No explicit dereferencing operator `*` should be used. Furthermore, to assign an address of a variable to a reference variable, no address-of operator `&` is needed.

For example,

```
/* References vs. Pointers (TestReferenceVsPointer.cpp) */
#include <iostream>
using namespace std;

int main() {
   int number1 = 88, number2 = 22;

   // Create a pointer pointing to number1
   int * pNumber1 = &number1;  // Explicit referencing
   *pNumber1 = 99;             // Explicit dereferencing
   cout << *pNumber1 << endl;  // 99
   cout << &number1 << endl;   // 0x22ff18
   cout << pNumber1 << endl;   // 0x22ff18 (content of the pointer variable - same as above)
   cout << &pNumber1 << endl;  // 0x22ff10 (address of the pointer variable)
   pNumber1 = &number2;        // Pointer can be reassigned to store another address

   // Create a reference (alias) to number1
   int & refNumber1 = number1;  // Implicit referencing (NOT &number1)
   refNumber1 = 11;             // Implicit dereferencing (NOT *refNumber1)
   cout << refNumber1 << endl;  // 11
   cout << &number1 << endl;    // 0x22ff18
   cout << &refNumber1 << endl; // 0x22ff18
   //refNumber1 = &number2;     // Error! Reference cannot be re-assigned
                                // error: invalid conversion from 'int*' to 'int'
   refNumber1 = number2;        // refNumber1 is still an alias to number1.
                                // Assign value of number2 (22) to refNumber1 (and number1).
   number2++;
   cout << refNumber1 << endl;  // 22
   cout << number1 << endl;     // 22
   cout << number2 << endl;     // 23
}
```

A reference variable provides a new name to an existing variable. It is *dereferenced implicitly* and does not need the dereferencing operator `*` to retrieve the value referenced. On the other hand, a pointer variable stores an address. You can change the address value stored in a pointer. To retrieve the value pointed to by a pointer, you need to use the indirection operator `*`, which is known as *explicit dereferencing*. Reference can be treated as a `const` pointer. It has to be initialized during declaration, and its content cannot be changed.

Reference is closely related to pointer. In many cases, it can be used as an alternative to pointer. A reference allows you to manipulate an object using pointer, but without the pointer syntax of referencing and dereferencing.

The above example illustrates how reference works, but does not show its typical usage, which is used as the function formal parameter for pass-by-reference.

2.4 Pass-By-Reference into Functions with Reference Arguments vs. Pointer Arguments

Pass-by-Value

In C/C++, by default, arguments are passed into functions *by value* (except arrays which is treated as pointers). That is, a clone copy of the argument is made and passed into the function. Changes to the clone copy inside the function has no effect to the original argument in the caller. In other words, the called function has no access to the variables in the caller. For example,

```
/* Pass-by-value into function (TestPassByValue.cpp) */
#include <iostream>
using namespace std;

int square(int);

int main() {
   int number = 8;
   cout <<  "In main(): " << &number << endl;  // 0x22ff1c
   cout << number << endl;         // 8
   cout << square(number) << endl; // 64
   cout << number << endl;         // 8 - no change
}

int square(int n) {  // non-const
   cout <<  "In square(): " << &n << endl;  // 0x22ff00
   n *= n;           // clone modified inside the function
   return n;
}
```

The output clearly shows that there are two different addresses.

Pass-by-Reference with Pointer Arguments

In many situations, we may wish to modify the original copy directly (especially in passing huge object or array) to avoid the overhead of cloning. This can be done by passing a pointer of the object into the function, known as *pass-by-reference*. For example,

```
/* Pass-by-reference using pointer (TestPassByPointer.cpp) */
#include <iostream>
using namespace std;

void square(int *);

int main() {
   int number = 8;
   cout <<  "In main(): " << &number << endl;  // 0x22ff1c
   cout << number << endl;   // 8
   square(&number);          // Explicit referencing to pass an address
   cout << number << endl;   // 64
}

void square(int * pNumber) {  // Function takes an int pointer (non-const)
   cout <<  "In square(): " << pNumber << endl;  // 0x22ff1c
   *pNumber *= *pNumber;      // Explicit de-referencing to get the value pointed-to
}
```

The called function operates on the same address, and can thus modify the variable in the caller.

Pass-by-Reference with Reference Arguments

Instead of passing pointers into function, you could also pass references into function, to avoid the clumsy syntax of referencing and dereferencing. For example,

```
/* Pass-by-reference using reference (TestPassByReference.cpp) */
#include <iostream>
using namespace std;

void square(int &);

int main() {
   int number = 8;
   cout <<  "In main(): " << &number << endl;  // 0x22ff1c
   cout << number << endl;  // 8
   square(number);          // Implicit referencing (without '&')
   cout << number << endl;  // 64
}

void square(int & rNumber) {  // Function takes an int reference (non-const)
   cout <<  "In square(): " << &rNumber << endl;  // 0x22ff1c
   rNumber *= rNumber;        // Implicit de-referencing (without '*')
}
```

Again, the output shows that the called function operates on the same address, and can thus modify the caller's variable.

Take note referencing (in the caller) and dereferencing (in the function) are done implicitly. The only coding difference with pass-by-value is in the function's parameter declaration.

Recall that references are to be initialized during declaration. In the case of function formal parameter, the references are initialized when the function is invoked, to the caller's arguments.